ME220 · INTRODUCTION TO MECHATRONICS

Assignment 1: Automatic Door

Released: 20.10.2026 • Due: **30.10.2026** • Total: 100 (+10 bonus)

Name: _____ Student no: _____

Overview

You will program the automatic door at a building entrance: when a person walks up the door swings open, it stays open while they pass, and it closes 3 s after the doorway is clear. A servo plays the door motor (0° = closed, 90° = open). You build and test everything in the **Wokwi** simulator with a Raspberry Pi Pico and MicroPython.

What you practise: digital outputs (LEDs), a debounced button, an analog input (potentiometer), PWM for a servo and a buzzer, the HC-SR04 ultrasonic sensor, and splitting a program into small functions.

What you start with: an empty Wokwi project. You build the circuit and write the whole program yourself.

What you hand in: your Wokwi project link and a short PDF report (at most 2 pages). Work individually. Expected effort: 5 to 8 hours.

1 Required behaviour

A person is *present* when the measured distance is below the threshold set by the potentiometer; no echo (out of range) counts as nobody there. The door repeats this sequence forever:

1. **Waiting** (door closed). About five times per second, print a status line. When a person is present *or* the button is pressed, go to step 2.
2. **Opening.** Two short beeps, green LED on, red LED off. The door swings open to 90° in steps of 3°, one step every 20 ms.
3. **Open.** The door stays open until there has been *nobody in the doorway for* 3 s *in a row*. A person seen again during those 3 s starts the count again.
4. **Closing.** The door swings shut to 0° in steps of 3°, one step every 20 ms. The red LED and the buzzer (2 kHz) switch on and off together every 0.2 s as a warning. **Before every step, check the sensor: if a person is present, stop where you are and open again** (step 2, starting from where the door stopped).
5. **Closed.** Buzzer off, red LED on, back to step 1.

Step	Door (servo)	Green LED	Red LED	Buzzer
Waiting / Closed	at 0°	off	on	silent
Opening, Open	opens to 90°	on	off	two short beeps (1 kHz) when opening starts
Closing	shuts to 0°	off	blinks (0.2 s)	beeps with the red LED (2 kHz)

Threshold: the potentiometer sets it from 10 cm (knob at minimum) to 100 cm (knob at maximum).

Button: the push-to-open button found at accessible entrances (for wheelchair users, or someone carrying boxes). One press gives exactly one opening; holding the button down must not open it again and again.

Status line: while waiting, your program must print this line in *exactly* this format (`dist=None` when there is no echo; `visitors=N` only if you do the bonus), so your result can be checked quickly:

```
dist=23.4 cm thr=55 cm visitors=0
```

2 Part A — Build the circuit 10 points

Create a new project at wokwi.com (Raspberry Pi Pico, MicroPython), add the parts with the + button and wire them exactly as below. The pins match the Pico Board v2, so the same code will later run on the real board.

Part (Wokwi name)	Connections
HC-SR04 ultrasonic sensor	VCC → 3V3, GND → GND, TRIG → **GP21**, ECHO → **GP20**
Servo	PWM → **GP11**, V+ → VBUS, GND → GND
Buzzer	one leg → **GP9**, other leg → GND
Potentiometer	VCC → 3V3, GND → GND, SIG → **GP26**
Pushbutton	one side → 3V3, other side → **GP6** (use `Pin.PULL_DOWN`)
Green LED + 330 Ω resistor	**GP5** → resistor → LED anode (A); cathode (C) → GND
Red LED + 330 Ω resistor	**GP4** → resistor → LED anode (A); cathode (C) → GND

Check before Part B: write a short test program that switches both LEDs on and prints `wiring OK`. In Wokwi, click the HC-SR04 while the simulation runs to set the distance with a slider (2–400 cm).

> On real hardware a classic HC-SR04 needs 5 V and its ECHO then outputs 5 V, which needs a voltage divider before GP20. Wokwi does not model voltages, so 3V3 is fine here.

3 Parts B to E — Write the program

You write the whole program yourself, using what you learned in the lectures (including the `map_range()` function). Split it into small functions with one job each. The names below are a suggestion; you may choose your own.

Suggested function	Its job
`get_distance_cm()`	one HC-SR04 measurement: distance in cm, or `None` when there is no echo
`get_threshold_cm()`	potentiometer → threshold of 10–100 cm
`person_present()`	`True` or `False`
`set_angle(angle)`	move the servo and remember where the door is
`beep(times)`	short beeps on the buzzer
`open_door()`	step 2 of the sequence
`wait_until_clear()`	step 3 of the sequence
`close_door()`	step 4; tells the caller whether the door closed fully or stopped for a person
`button_pressed()`	`True` once per press

Main loop, in words: forever — print the status line; if a person is present or the button was pressed, open the door, wait until the doorway is clear, then close it; wait 0.2 s. If closing stopped for a person, the next pass of the loop simply opens the door again.

Part B — Sensing 20 points

1. Measure the distance: a 10 µs trigger pulse, then `time_pulse_us(echo, 1, 30000)`. A negative result means no echo; treat it as `None`, never as a distance.

2. Read the threshold from the potentiometer: 10 cm at minimum, 100 cm at maximum.
3. A person is present when the distance is not `None` and below the threshold.
4. While waiting, print the status line in the required format about five times per second.

Part C — Opening the door — 20 points

1. Servo: clamp the angle to 0°–180°, convert a 0.5–2.5 ms pulse to `duty_u16` at 50 Hz, and keep the door's current angle in a variable (e.g. `door_angle`).
2. Buzzer: a beep is 1 kHz for 0.1 s, then 0.1 s of silence.
3. Opening: two beeps, LEDs as in the table, then move the door **from its current angle** up to 90° in steps of 3°, waiting 20 ms after each step.

Part D — Waiting and closing — 30 points

1. Waiting: check for a person every 0.1 s and count the checks with nobody there. Continue when the count reaches 30 (3 s); set it back to 0 whenever a person is seen. A 1 s gap must not close the door. Do not use one long `sleep(3)`.
2. Closing: move the door from its current angle down to 0° in steps of 3°, 20 ms apart, with the red LED and the buzzer (2 kHz) switching on and off together every 10 steps.
3. **Safety:** before every closing step, check for a person. If someone is there, switch the buzzer off and stop closing at once, and let the main loop know the door did not close.

Why opening must start from the door's current angle

If closing stops half-way at, say, 45° and your opening loop always starts at 0°, the door first slams *shut* to 0° — into the person — and only then opens. Start the loop at the angle where the door is now, e.g. `for angle in range(door_angle, 91, 3):`

Part E — Manual open button — 10 points

1. The button opens the door only at the moment it goes from 0 to 1. Remember the last value you read; when the value changes, wait 0.02 s for the bounce to settle.
2. Holding the button for several seconds must give exactly one opening.

Bonus — up to 10 points

Count the visitors: add one each time the door closes completely, and add `visitors=N` to the end of the status line.

4 Test checklist

Run every test in Wokwi before you submit; the grader runs the same ones. Unless stated, the knob is in the middle (threshold ≈ 55 cm). Set distances by clicking the HC-SR04 while the simulation runs.

#	Do this	Expected result	✓
1	Start with the sensor at 200 cm	Door at 0°, red on, green off; status lines printed	☐
2	Set 30 cm	Two beeps, door opens smoothly to 90°, green on	☐
3	Keep 30 cm for 10 s	Stays open	☐
4	Set 200 cm and wait	Stays open ≈ 3 s, then closes; red LED and buzzer blink while it moves	☐
5	While open: set 200 cm for about 1 s, then back to 30 cm	Does not close	☐
6	While it is closing, set 30 cm	Door stops and opens again from where it stopped, without slamming to 0° first	☐
7	Knob at minimum, person at 30 cm	Stays closed (threshold 10 cm)	☐
8	Knob at maximum, person at 80 cm	Opens (threshold 100 cm)	☐
9	Set 400 cm	Treated as nobody there; no error	☐
10	Nobody there, press the button once	Opens, then closes by itself after ≈ 3 s	☐
11	Nobody there, hold the button down for 6 s	Opens and closes once only	☐

5 Report, submission and grading

Report (PDF, at most 2 pages)

1. Draw a flowchart of your program: the main loop, the waiting step and the closing step.
2. Why does the door wait for 3 s *in a row* with nobody there before closing, instead of closing the moment the distance rises above the threshold?
3. Why must the door check the sensor at every closing step? What could happen with a real door if it did not?
4. The door moves 3° every 20 ms. How long does a full opening take? Show the calculation.
5. Press the button while the door is already open and waiting. What happens, and why? (Think about which line of code the Pico is running at that moment.)

How to submit

- In Wokwi, press **Save**, then **Share** and copy the link. Check that the link opens in a private browser window.
- Upload the report PDF with the Wokwi link written on its first line.

Grading

Part	What is checked	Points
A — Circuit	All parts wired to the listed pins; LEDs have resistors	10
B — Sensing	Correct distance, timeout handled (test 9), threshold from the knob (tests 7, 8)	20
C — Opening	Clamped servo function, beeps, LEDs, smooth opening from `door_angle` (test 2)	20
D — Waiting and closing	3 s count restarts (test 5), warning blink, safety stop and reopen (test 6)	30
E — Button	Opens on a press (test 10); one opening per press (test 11)	10
Report and code	Flowchart, answers 2–5; program split into functions, readable, with comments	10
Bonus	Visitor counter in the status line	+10
Total		**100 (+10)**

A program that stops with an error in any test loses the points of that part.